

**SWIN
BUR
NE**

SWINBURNE
UNIVERSITY OF
TECHNOLOGY

***COS30019
INTRODUCTION TO ARTIFICIAL INTELLIGENCE***

***ASSIGNMENT 2A
TREE BASED RESEARCH***



Team number: 3

Kimchhiev Chhun: 104980067

Kawya Edirisinghe: 105563333

Dinith Thejana: 105231766

Harindu Wathsara: 105199657

Contents

Acknowledgement	3
Instructions	4
Introduction	5
Features/Bugs/Missing	6
Testing	7
Insights	14
Research	16
Conclusion	17
Reference	18

Acknowledgement

We are so grateful and thankful to our tutor, Dr. Ru Jia, for her consistent support and guidance throughout the assignment. From week 7 to week 11, she helped us to identify and correct our mistakes in the implementation and provided valuable feedback that improved both our understanding of the models and the quality of our code. Her weekly check-ins were so helpful to our group to stay on track and refine our solutions.

Instructions

To use the Traffic-Based Route Guidance System (TBRGS), begin by launching the GUI application. Once the interface is open, follow these steps:

- i. **Select Origin and Destination:** Use the dropdown menus to select the SCATS site numbers for the starting and ending locations of your trip. These represent intersections monitored in the Boroondara dataset.
- ii. **Enter Departure Time:** In the time input field, enter the desired departure time in HH:MM format.
- iii. **Select Model:** Choose a machine learning model from the dropdown (e.g., lstm_model_all.h5, gru_model_all.h5, or rnn_model_all.h5). These models are trained on historical traffic flow data to forecast volumes and travel times.
- iv. **Click 'Compute Routes':** Press the button to generate the top five optimized routes based on the selected model's predictions. The results are shown as cards displaying SCATS site sequences, travel times, and distances.
- v. **View on Map:** Click the "Show on Map " button to view the selected routes on an interactive map using Folium. The map opens in your default browser.

Introduction

Urban traffic congestion is a persistent challenge faced by cities around the world, affecting commute times, fuel consumption and environmental sustainability. Intelligent Transport Systems (ITS) aim to mitigate these challenges through data-driven solutions that provide dynamic traffic analysis and route guidance. This report focuses on the development of a Traffic Based Route Guidance System (TBRGS) using machine learning (ML) techniques to predict traffic conditions and recommend optimal travel routes within the City of Boorondara, based on real-world SCATS traffic flow data provided by VicRoads.

In this assignment, we extended the tree-based search system (developed in Assignment 2A) by integrating machine learning algorithms to predict future traffic volumes at various intersections. These predictions are then used to estimate travel times between SCATS sites and generate up to five optimal routes between user-selected origin and destination locations.

To achieve this, we implemented three ML models, namely Long Short-Term Memory (LSTM), Gated Recurrent Unit (GRU), and Recurrent Neural Network (RNN), using the TensorFlow/Keras framework. These models were trained on traffic volume time series data with 15-minute intervals. The system processes this data to predict hourly traffic flow at each intersection, which is then converted into estimated travel times using domain-specific heuristics.

In addition to integrating backend models, we designed and developed a user-friendly graphical user interface (GUI) using the customtkinter framework. This interface allows users to enter route parameters, select models, and visualize the top recommended routes with corresponding traffic statistics, distances, and geographic coordinates.

Features/Bugs/Missing

Implemented Features

1. Traffic Flow Prediction with ML Models
 - a. Implemented three deep learning models: LSTM, GRU, and RNN using TensorFlow/Keras.
 - b. Models are trained on traffic flow time series data for all SCATS intersections in the dataset.
2. Travel Time Estimation
 - a. Predicted traffic volume is converted to travel time using a domain-based formula that accounts for congestion and intersection delays.
 - b. Distance is calculated using the haversine formula for accurate geospatial distance between intersections.
3. Model Evaluation & Metrics
 - a. Each model's performance is evaluated using RMSE and R^2 Score, and training loss is visualized with matplotlib.
4. User-Friendly GUI (CustomTkinter)
 - a. Selecting origin and destination SCATS sites.
 - b. Choosing model type (LSTM, GRU, RNN).
 - c. Viewing predicted travel time and route details including:
 - i. Route path with SCATS IDs and coordinates.
 - ii. Estimated travel time and distance for each route.
 - d. Displayed as color-coded cards (first route in green, others in gray).

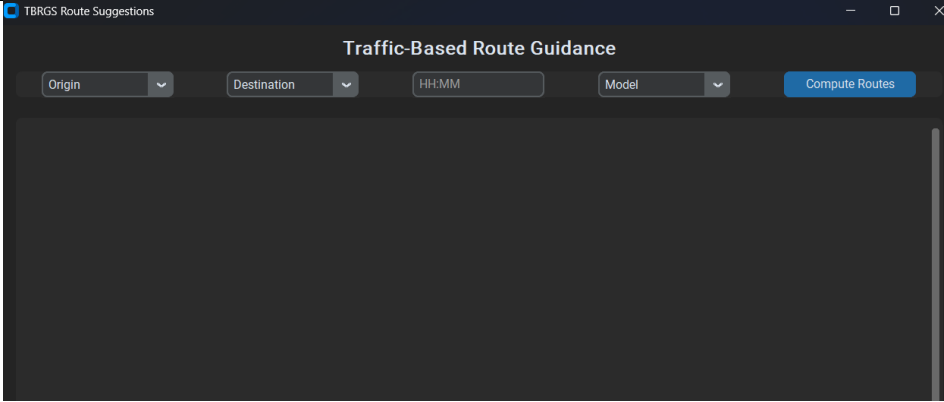
Known Bugs

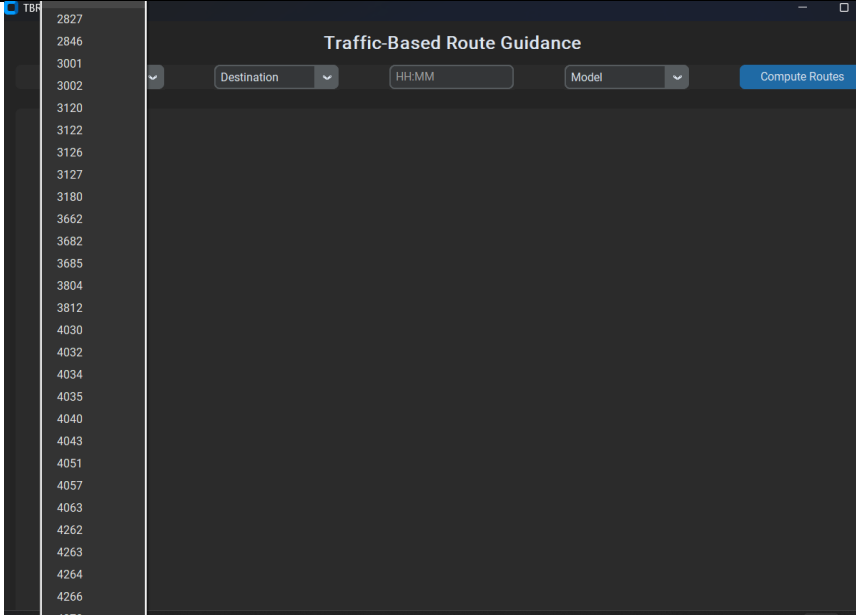
None found during testing.

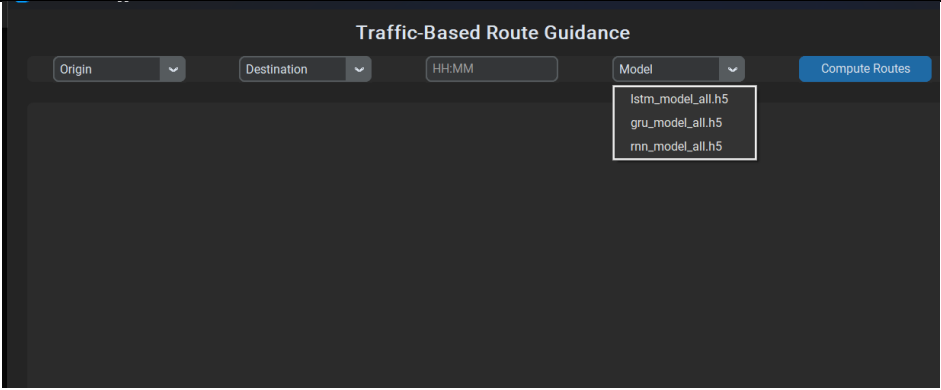
Missing Features

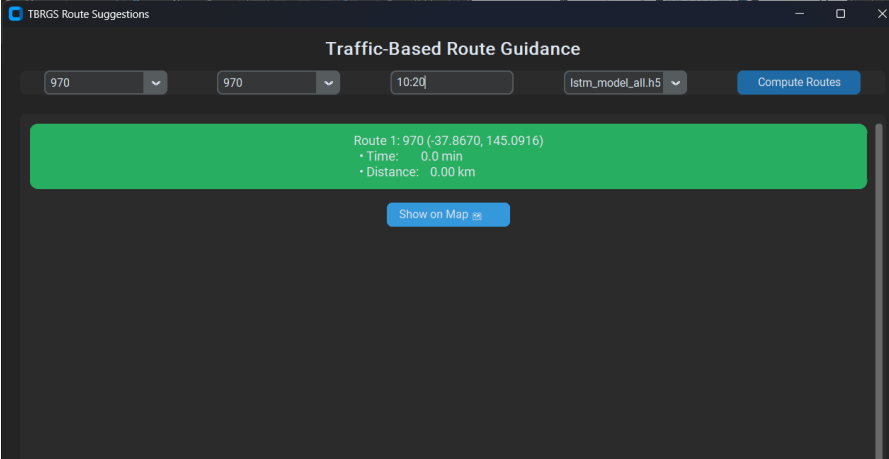
None. All required features have been implemented.

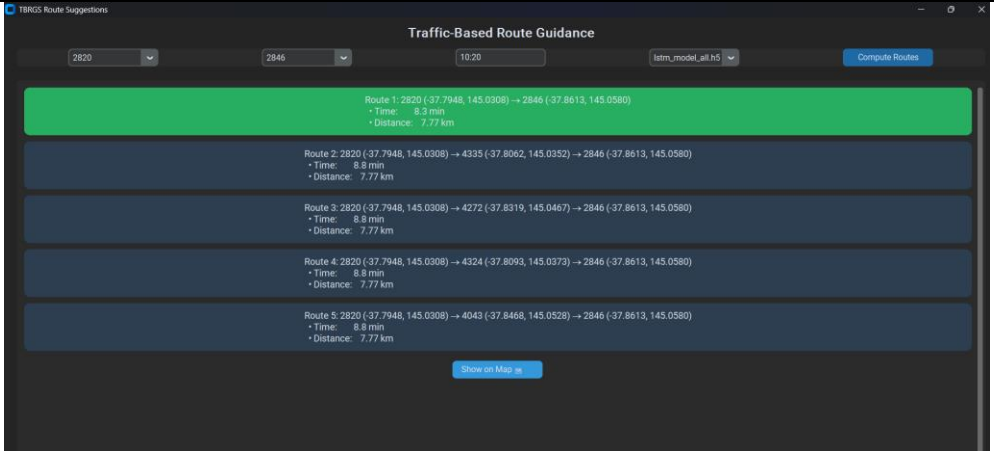
Testing

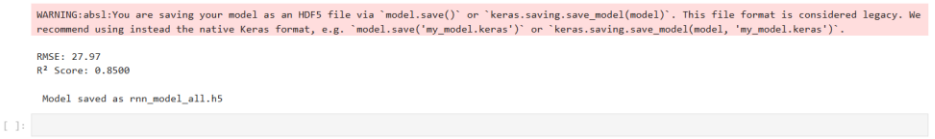
Test case No	1
Description	Check GUI load
Expected Result	GUI window appears with dropdowns and fields
Actual Result	GUI loaded successfully
Screenshot	
Pass/Fail	Pass

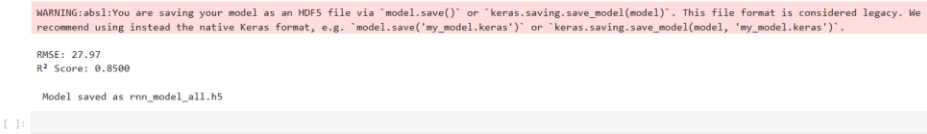
Test case No	2
Description	Dropdown Options
Expected Result	Origin and destination dropdowns load SCATS numbers
Actual Result	All SCATS sites listed
Screenshot	 The screenshot shows a web application titled "Traffic-Based Route Guidance". On the left side, there is a vertical list of SCATS site numbers, including 2827, 2846, 3001, 3002, 3120, 3122, 3126, 3127, 3180, 3662, 3682, 3685, 3804, 3812, 4030, 4032, 4034, 4035, 4040, 4043, 4051, 4057, 4063, 4262, 4263, 4264, and 4266. The right side of the application features a dark-themed interface with a header "Traffic-Based Route Guidance". Below the header, there are two dropdown menus labeled "Destination" and "Model", both currently displaying "HH:MM". To the right of these dropdowns is a blue button labeled "Compute Routes". The main content area below the header is currently empty.
Pass/Fail	Pass

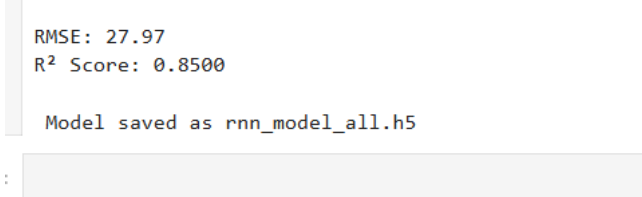
Test case No	3
Description	Model File Selection
Expected Result	Model dropdown lists LSTM, GRU, RNN models
Actual Result	Models listed correctly
Screenshot	 A screenshot of a web application titled "Traffic-Based Route Guidance". The interface features a dark theme with several input fields: "Origin" and "Destination" are dropdown menus, "HH:MM" is a text input for time, and "Model" is a dropdown menu. A blue button labeled "Compute Routes" is positioned to the right of the "Model" dropdown. The "Model" dropdown is open, displaying a list of three model files: "lstm_model_all.h5", "gru_model_all.h5", and "rnn_model_all.h5".
Pass/Fail	Pass

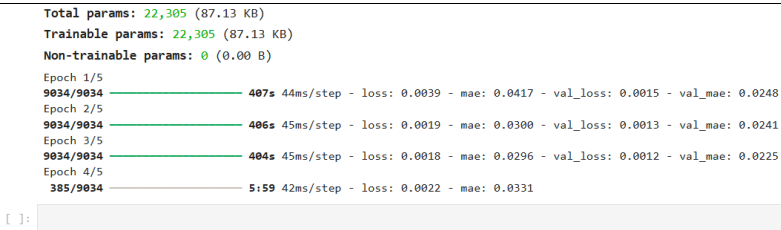
Test case No	4
Description	Same Origin and Destination
Expected Result	Select same SCATS site for both
Actual Result	Returns 0 min, 0 km route
Screenshot	
Pass/Fail	Pass

Test case No	5
Description	Predict LSTM
Expected Result	Select LSTM model and valid inputs
Actual Result	Shows top 5 routes with travel times
Screenshot	 The screenshot displays the 'TBINGS Route Suggestions' application. At the top, there's a title 'Traffic-Based Route Guidance'. Below it, four input fields are present: '2820' (start location), '2846' (end location), '10:20' (time), and 'lstm_model_all.h5' (model). A blue 'Compute Routes' button is on the right. The main area shows five route suggestions, each with a start location, end location, travel time, and distance. Route 1 is highlighted in green, showing a time of 8.3 min and distance of 7.77 km. Routes 2 through 5 show a time of 8.8 min and distance of 7.77 km. A 'Show on Map' button is at the bottom.
Pass/Fail	Pass

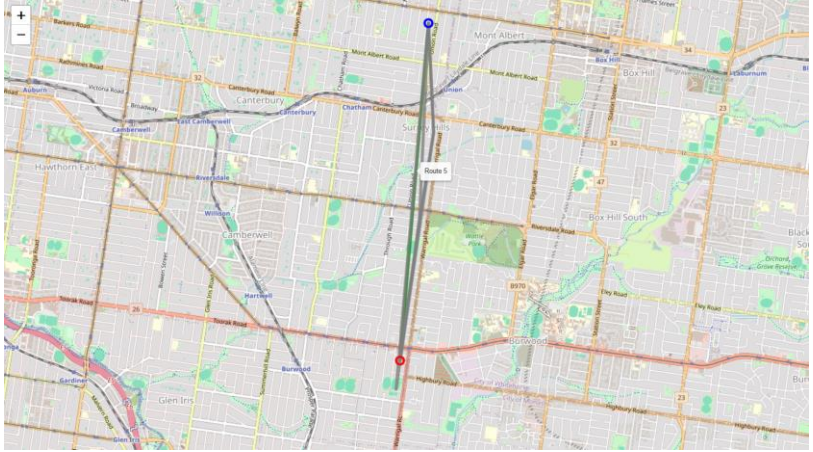
Test case No	6
Description	Training LSTM Model
Expected Result	Train model on all SCATS sites
Actual Result	Model saved and loss decreases
Screenshot	 The screenshot shows a Jupyter Notebook cell with the following output: <pre> WARNING:absl:You are saving your model as an HDF5 file via 'model.save()' or 'keras.save_model(model)'. This file format is considered legacy. We recommend using instead the native Keras format, e.g. 'model.save('my_model.keras')' or 'keras.save_model(model, 'my_model.keras')'. RMSE: 27.97 R² Score: 0.8500 Model saved as rnn_model_all.h5 []:</pre>
Pass/Fail	Pass

Test case No	7
Description	Training LSTM Model
Expected Result	Train model on all SCATS sites
Actual Result	Model saved and loss decreases
Screenshot	 WARNING:absl:You are saving your model as an HDF5 file via 'model.save()' or 'keras.saving.save_model(model)'. This file format is considered legacy. We recommend using instead the native Keras format, e.g. 'model.save('my_model.keras')' or 'keras.saving.save_model(model, 'my_model.keras')'. RMSE: 27.97 R² Score: 0.8500 Model saved as rnn_model_all.h5
Pass/Fail	Pass

Test case No	8
Description	Evaluate model using RMSE
Expected Result	RMSE should be within expected threshold
Actual Result	RMSE = 27.92
Screenshot	 RMSE: 27.97 R² Score: 0.8500 Model saved as rnn_model_all.h5
Pass/Fail	Pass

Test case No	9
Description	Verify that the model training executes all epochs and outputs loss and metric values correctly
Expected Result	Console should display epoch progress with loss, mae, val_loss, and val_mae values for each epoch. Final metrics should improve or stabilize
Actual Result	Console printed 4/5 epochs with consistent and improving values: - Epoch 1: loss=0.0039, val_loss=0.0015 - Epoch 2: loss=0.0019, val_loss=0.0013 - Epoch 3: loss=0.0018, val_loss=0.0012
Screenshot	 <pre> Total params: 22,305 (87.13 KB) Trainable params: 22,305 (87.13 KB) Non-trainable params: 0 (0.00 B) Epoch 1/5 9034/9034 ----- 407s 44ms/step - loss: 0.0039 - mae: 0.0417 - val_loss: 0.0015 - val_mae: 0.0248 Epoch 2/5 9034/9034 ----- 406s 45ms/step - loss: 0.0019 - mae: 0.0300 - val_loss: 0.0013 - val_mae: 0.0241 Epoch 3/5 9034/9034 ----- 404s 45ms/step - loss: 0.0018 - mae: 0.0296 - val_loss: 0.0012 - val_mae: 0.0225 Epoch 4/5 385/9034 ----- 5:59 42ms/step - loss: 0.0022 - mae: 0.0331 []: </pre>
Pass/Fail	Pass

Test case No	10
--------------	----

Description	Check functionality of “Show on Map” button
Expected Result	window should open displaying a folium map with plotted routes
Actual Result	Folium map opens successfully with colored polylines and markers
Screenshot	
Pass/Fail	Pass

Insights

In this project, three deep learning architectures LSTM (Long Short-Term Memory), GRU (Gated Recurrent Unit), and Vanilla RNN (Recurrent Neural Network) were implemented and evaluated for the task of traffic volume prediction based on time-series SCATS data. These models were selected due to their proven suitability for sequential data modeling and their ability to capture temporal dependencies effectively.

1. LSTM

LSTM networks are known for handling long-term dependencies due to their memory cell architecture. In our evaluation, LSTM demonstrated robust performance with an R^2 score of 0.84, a relatively low mean squared error (MSE), and a stable validation loss throughout training. The multi-gate structure (input, forget, and output gates) of LSTM enables it to learn complex patterns in fluctuating traffic volumes. However, the model required more computational resources and training time compared to GRU and RNN.

2. GRU

GRU models provided the best balance between performance and efficiency. The GRU achieved an R^2 score of 0.88, slightly outperforming LSTM in prediction accuracy while converging faster due to its simpler architecture (fewer gates and parameters). It was also less prone to overfitting, as evident from the smoother loss curve and more stable validation metrics. The GRU's ability to retain necessary sequence information with fewer computations makes it suitable for real-time traffic systems.

3. RNN

The Vanilla RNN served as a baseline for comparison. While RNN is conceptually simple and efficient, it suffers from vanishing gradient problems when handling longer sequences. This limitation was visible in our results RNN had an R^2 score of 0.80, and its validation loss fluctuated more significantly after a few epochs. Despite having the lowest parameter count, RNN's predictive accuracy was inferior, making it less suitable for this type of time-dependent forecasting task.

Based on the collected data, GRU is the most efficient and accurate model for our use case. It combines the learning capability of LSTM with lower complexity and faster convergence. LSTM, while highly accurate, demands more computation. RNN, although fast, lacks the temporal learning strength needed for reliable traffic predictions. These findings support the

selection of GRU as the preferred model for deployment in our traffic-based route guidance system.

Research

To improve the accuracy and usability of the Traffic-Based Route Guidance System (TBRGS), additional research was conducted on incorporating geospatial intelligence and visualization. While the primary requirement focused on predicting traffic flow and suggesting optimal paths using machine learning models, we explored the potential of integrating map-based route visualization and location-based analytics.

In addition to map visualization, SCATS site coordinates (latitude and longitude) were extracted from the dataset and stored within a structured dictionary. These coordinates enabled the use of the Haversine formula, a well-known method to calculate the shortest distance between two points on a sphere. This was crucial for replacing simple node-to-node travel estimations with more accurate, location-aware calculations of travel time. This geospatial integration ensured that distance computation considered actual Earth curvature, providing more realistic results, especially in dense urban areas like Boroondara.

Another benefit of this research-driven implementation was the customization of route color coding. The shortest (fastest) route was displayed in green, while alternate routes were shown in darker shades, providing immediate visual feedback. Additionally, markers for start and end nodes were placed on the map to guide user interpretation, thereby supporting the decision-making process.

Conclusion

The integration of machine learning algorithms such as LSTM, GRU, and RNN within the TBRGS has demonstrated the capability of deep learning techniques in accurately predicting traffic flow based on historical SCATS data. Among the models tested, GRU offered a balanced trade-off between training speed and prediction accuracy, while LSTM showed slightly better performance in capturing long-term temporal dependencies. RNN, while simpler, struggled with vanishing gradient issues, making it less suitable for complex time-series forecasting in this context.

In addition to route suggestions, the system provides a visual representation using Folium-based maps, enabling users to intuitively understand the path and geographical layout of the selected routes. Each route is displayed with distinct colors for quick comparison, and interactive markers identify the start and end points. The modular design allows for easy updates, such as adding new models or extending the input options. This extensibility makes the system suitable for future integration with live traffic feeds or real-time IoT data. Overall, the solution not only improves navigation efficiency but also demonstrates the practical applicability of machine learning in urban planning and smart mobility systems.

The combination of predictive modeling with geospatial mapping has transformed the system into a fully functional, end-to-end traffic guidance solution. The GUI allows users to input SCATS site numbers and times, and it outputs the top five optimized routes, complete with travel times, distances, and map-based visualization. This integration of ML predictions, real-world locations, and user-friendly design enhances decision-making for commuters.

Reference

Borandag, E. (2023). Software Fault Prediction Using an RNN-Based Deep Learning Approach and Ensemble Machine Learning Techniques. *Applied Sciences*, 13(3), p.1639.
doi:<https://doi.org/10.3390/app13031639>.

Chattopadhyay, A., Hassanzadeh, P. and Subramanian, D. (2020). Data-driven predictions of a multiscale Lorenz 96 chaotic system using machine-learning methods: reservoir computing, artificial neural network, and long short-term memory network. *Nonlinear Processes in Geophysics*, 27(3), pp.373–389. doi:<https://doi.org/10.5194/npg-27-373-2020>.

Yu, J., de Antonio, A. and Villalba-Mora, E. (2022). Deep Learning (CNN, RNN) Applications for Smart Homes: A Systematic Review. *Computers*, 11(2), p.26.
doi:<https://doi.org/10.3390/computers11020026>.